Shopping Cloud

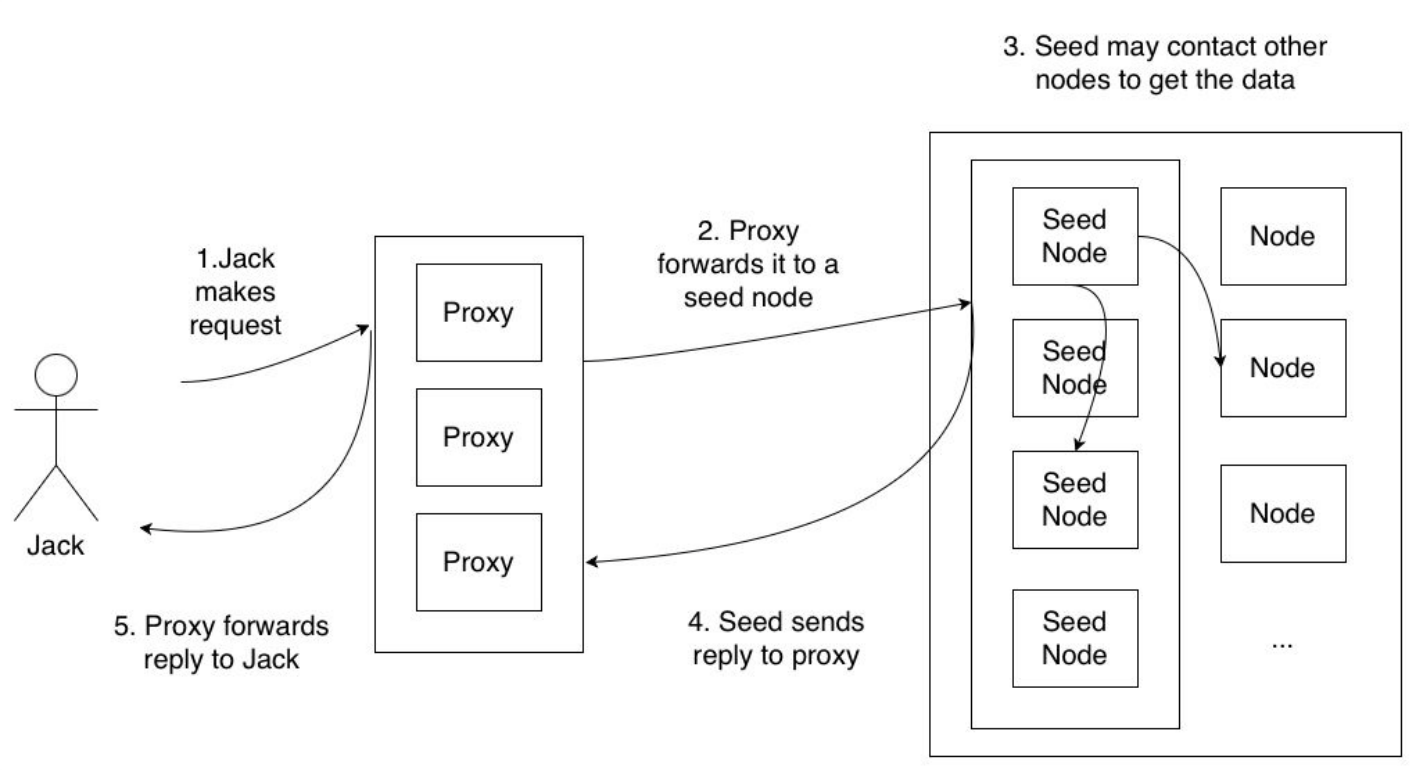
Large Scale Distributed Systems

Turma 7 Grupo 7
Nuno Rios - up202206272
Rodrigo Ribeiro - up202206396
Rodrigo Resende - up202108750

Introduction

Shopping Cloud is a distributed and scalable shopping list application designed following local-first principles. The system supports concurrent updates by multiple users while ensuring high availability, fault tolerance, and horizontal scalability. The project explores key concepts from large-scale distributed systems, such as sharding, replication, eventual consistency, and conflict-free data types (CRDT).

System Architecture Diagram



Implementation & Deployment Stack

The system was implemented in Java;

Git/ Gitlab was used for version control;

Maven (and Maven modules) used as the build system with 4 modules: **common, client, proxy and server**, where common holds code common to all other modules;

Automation scripts written to aid in deployment and testing;

TMUX used to allow for a single terminal to deploy multiples node/ proxy instances.

Storage and Serialization

Storage is simple and only provides 3 main operations: **get, put and remove**.

Values are stored as Json strings; We used the Jackson library to help with this.While this is harder to implement than simply implementing the **Serializable** interface, it is 1. human readable and 2. compatible with other languages, so we could easily add a HTML frontend without changing the backend.

Storage itself is an interface, and we provide a default **FileStorage** implementation using files, since it is the simplest. The system is made to easily support other storage systems, such as DB-based, without modifying the underlying implementation.

CRDT Implementation

All Shopping Lists are represented using Conflict-free replicated data types to ensure eventual consistency. In fact, the ShoppingList class itself is a CRDT. All CRDTs have a merge function.

We can divide the CRDTs implemented into 3 categories: **Registers**, which hold specific values, **Counters**, representing growing integers and **Collections**, such as sets and maps.

Counters:

- **GCounter**, a grow-only counter which holds the count for each replica (figure below)
- **PNCounter**, which can increase or decrease and is implemented internally using 2 GCounters, one for increments and another for decrements.

```
private final Map<UUID, Integer> count;
```

CRDT Implementation

Registers:

- **LWWRegister**, a register that holds a value of a specified type along with a timestamp.

A timestamp is made up of a replica/ node ID and a count. Merge logic of the LWWRegister involves comparing timestamps-the register takes the value of the one its merging with if its current timestamp **“happened before”** the other.

```
@Override
public void merge(LWWRegister<T> other) {
    if (timestamp.happenedBefore(other.timestamp)) {
        value = other.value;
        timestamp = other.timestamp;
    }
}
```

A timestamp is **arbitrarily** considered to “happen before” another if its count < other.count or, if the counts are equal, if the nodeID is alphabetically < than other.nodeID.

```
public boolean happenedBefore(Timestamp other) {
    return count < other.count ||
            (count == other.count && node.compareTo(other.node) < 0);
}
```

CRDT Implementation

```
@JsonProperty("set")
final Map<T, Map<UUID, Integer>> items = new HashMap<>();

@JsonProperty("version")
final Map<UUID, Integer> versionDots = new HashMap<>();
```

GSet- a simple single-only, set. No difference than a normal Java set, but disallows removals.

AWSet (Add-wins set)- represents a Java set. By far the most difficult to implement. We did not follow the “tombstone/removed set” approach, as memory grows linearly with each operation (each removal would require adding the removed item to the tombstone).

Instead, we use **dots**. A dot is a pair <node, count> and acts as a vector clock.

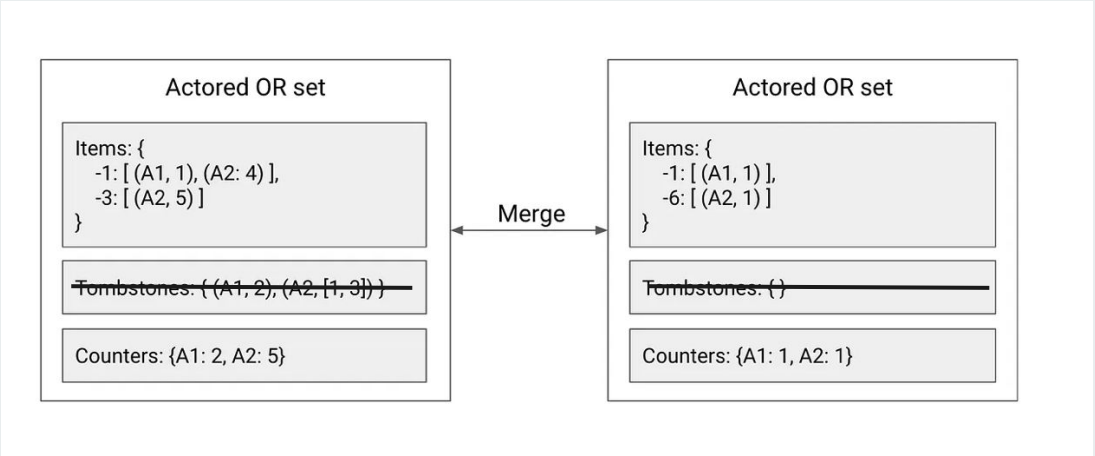
Thus, our AWSet holds the items within it and each of their dots, plus global versionDots map containing the most recent count for each node.

It is considered an “Add-wins” set because if an add and remove operation happen concurrently, the add takes precedence.

CRDT Implementation

Merge function of the AWSet involves comparing dots to deduce items that have been added and removed from the set.

Figure below taken from on Medium.com, where we discovered this AWSet design.



CRDT Implementation

```
final AWSet<AWMapEntry<K, V>> awSet = new AWSet<>();
```

AWMap, add-wins Map. Most operations are O(1), except for get(), which is O(n). Value type must be a CRDT.

Represented internally as an AWSet, with entries of type **AWMapEntry**.

Each AWMapEntry is a pair <Key, Value>, but its comparison logic ignores the values. Meaning if we compare (==) two AWMapEntries, they are equal if their keys are equal.

It is beneficial compared to using a set for keys + a map of key to value because **1.** we use less memory, no need to hold keys twice and **2.** the merge logic is simpler, we merge the set, and then the values.

```
// Merge a AWMap with this one
@Override
public void merge(AWMap<K, V> other) {
    // Merge key set first
    awSet.merge(other.awSet);

    // Merge values
    for (var otherEntry : other.awSet.value()) {
        var entry = this.get(otherEntry.key());
        if (entry != null)
            entry.merge(otherEntry.value());
    }
}
```

CRDT Implementation

ItemQuantity-A PNCounter along with a max count, enforced only locally.

ShoppingList-Contains various CRDTs mentioned earlier. Its merge function simply merges all of its CRDT attributes.

```
@JsonProperty private final UUID id;
@JsonProperty private final LWWRegister<String> name;
@JsonProperty private final AWMap<String, ItemQuantity> items;
```

Client-side - Local-first Design

Application follows the **“local-first”** principle. Updates are stored locally first.

To interact with the cloud, you use the /push and /pull commands. We did not waste much time on the client-side, so there are no automatic, periodic syncs.

To interact with the cloud, the client contacts a list of “static proxies”-we will talk about proxies in the following slides-and tries each one sequentially, in random order, until receiving a response.

Proxy Layer

As in any other system, proxies in web infrastructure are simply intermediaries between clients and servers. They are usually used for 1. security isolation and 2. load balancing.

We don’t really need proxies for load balancing-contacting a random seed node directly would be enough, since, either way, we will end up contacting the specific servers associated with the requested key.

However, for our use case, proxies are still useful for security isolation-they act as a buffer/ controlled boundary between clients and servers, reducing the attack surface. If proxies are breached, the server implementation “should” remain intact.

Consistent hashing- Hash Ring

- **XXHash** (non-cryptographic) algorithm is used for hashing. It is deterministic, uniform and very efficient, making it suitable for high-throughput systems.

- Virtual nodes are used to improve data distribution and reduce imbalance when nodes join or leave the system.

- Java TreeMap used as the backbone of the hash ring. TreeMap is implemented as a Red-black tree, so keys are kept in sorted order and lookups are O(log N). Ordering is essential to “move clock-wise” to find the node responsible for a key. TreeMap provides **map.ceilingKey(key)** just for this.

- Node positions are computed deterministically from the node ID (unlike Dynamo). So even if we “kill” the ring and restart the nodes, the data distribution won’t change. Computing it in a random fashion is overkill for our use case, especially since we would need to send the node hashes during ring exchanges.

Consistent hashing- Distributed Hash Ring

- Since our ring state is to be distributed across nodes, we implemented an elegant CRDT version of our hash ring entitled **DistributedHashRing**. It contains a regular HashRing along with an AWMap<node ID, node Object> containing the members of the ring.

- When exchanging rings, we simply exchange the AWMap. The ring can be reconstructed purely from its members, since, once again, node positions are deterministic.

- Since our AWMap implementation grows with the number of items in it and does not contain a tombstone, this is an entirely valid approach and simplifies converging the ring state during gossip (it just calls the merge function).

Communication

ZeroMQ was chosen as the communication framework; Important to mention that it is not simply a library but a framework, and so presents a set of patterns (and anti-patterns);

Anyone **making** a request using a **REQ** socket (usually in a relaxed state, so that a reply is not strictly required).

Our objective was for each node (and proxy) to be able to handle multiple requests concurrently, while not exceeding system resources.

The main challenge was a mix of “threads and sockets”. Sockets cannot be shared across threads, yet if we don’t use threads we cannot handle multiple requests concurrently. Locks are not an option, so we need a socket per thread.

However, we cannot have infinite threads (even if virtual), since we don’t have infinite system resources (not only threads but especially sockets, as these need file descriptors)

Communication

Solution: Have a main thread (the **Manager**) with a **ROUTER** socket that binds to the node address and polls external requests. Then, use a fixed, parameterizable number of threads, we'll call them **Workers**, each with its own socket for requests to other nodes. To connect the manager and the workers, we use a **ZMQ inproc socket,** designed especially for this.

External requests are routed from the manager to a "**free**" worker and that worker then becomes "**busy**". After processing the request, it sends the reply back to the manager, which forwards it to its sender and deems the worker as free.

A **ZMQ poller** is used by the Manager used to periodically poll messages from the exterior and from the workers.

Bootstrapping

- When a node starts up, it first contacts one of the **“seed nodes”** (using a **STATE** request) to get the ring. It then adds itself to the ring, which will then be gossiped (we will mention gossip later);

- If no nodes are up, the first node to bootstrap and the only node allowed to create the ring is the **first** of the seed nodes. This is to avoid a common problem in bootstrapping, where two nodes can bootstrap concurrently and we end up with two separate rings.

- The first of the seed nodes should **not** be considered a “master seed”, as it is only responsible for creating the ring. After the ring is created, any other node can be contacted for the purpose of getting the ring. If the first of the seeds goes down, this is no big deal (as long as there are others in the ring, of course).

Requests and Request Handling

Each request has its own handler class (extends Request) to keep the code scalable and to avoid a single class with 1000+ lines.

We currently support the following requests:

GET, PUT (to coordinate a get or put operation)

REPLICA_GET, REPLICA_PUT (to simply store or get the value from storage)

STATE (mentioned earlier)

GOSSIP (to be mentioned)

GET and PUT operations have a fixed timeout.

Requests and Request Handling

Quorums are used for availability + consistency. Since we are using CRDTs, eventual consistency is guaranteed, so we only need quorums for availability.

But what does this entail when it comes to requests?

A GET request simply gets the requested value, so the point of the quorum is purely for consistency, therefore we do not need it. We only need a single node to give us the value, that's it. We try each one sequentially until we get a successful reply.

The PUT request is different. The quorum here is for both consistency and availability (through replication). We only need the latter, so we will **replicate the PUT to all R replicas, however**, we only require success from 1 replica to return a success to the client.

This ties into failure detection, which will will mention later.

Identifying nodes

Nodes are identified by a unique id.

Their address is held in a LWWRegister so that a node is allowed to change its network address dynamically (often happens in distributed systems) without leaving the ring.

```
public class Node implements ICRDT<Node> {
    @JsonProperty
    private UUID id;

    // Node's zmq network address stored as a LWWRegister to
    // allow for a node to dynamically change its net address
    // without leaving the ring
    @JsonProperty
    private LWWRegister<String> address;

    @JsonProperty
    private final LWWRegister<NodeStatus> status;
```

Gossiping

A gossip protocol is used so that all nodes have roughly the same view of the system without a central authority. Specifically, given our CRDT implementation, all nodes are guaranteed to **eventually** have the same view of the ring.

Every second, each node “gossips” the ring to a random node in the ring.

Gossip is crucial for the two features will be mention next: **data transfers and fault detection**.

Data Transfers

When a node joins or leaves the ring, this implies a change in key distribution, and so, in case of a join, the new node will need to get the keys it is now responsible for. Conversely, if a node leaves, its keys need to end up with its new owners. **But how?**

We implement data transfers using a **PUSH-based approach** (like Dynamo). When a node joins, the news will be gossiped and eventually reach all nodes. When other node receive this gossip and notice a change in the ring, they detect that a range of keys they possess now have a new owner, and will thus stream the data to it.

A **PULL-based approach** (like Cassandra does), where data transfers are part of the bootstrapping, would have been valid, but is not strictly necessary for our needs.

Failure Detection

We need a mechanism to detect when nodes are down.

In our implementation, a node can either be **UP** or **DOWN**. A node is considered to be “down” when it fails to respond to a request, specifically:

1. When it fails to respond to a GOSSIP or
2. When it fails to respond to a GET or PUT request

From there, we avoid contacting the node until we know for sure it is UP.
But how do we know this?

It’s simple! Every time a node gossips, it makes sure to set itself as UP. So, when the failed node recovers (either after some network lag or a full reboot), it will gossip the ring and its state and UP. When other nodes receive and merge the new status (representing by an LWWRegister), the node that was down is now seen as up.

Failure Detection–Hinted Handoff

If we receive a GET or PUT request targeting a node that is currently down, we store the data in local storage, in a separate directory–this is known as a **hint**.
When the node comes back up, we deliver the hints to it.

A node with a hint may serve it to the client if they request that key.

Internally, hint storage is managed by a **HintStorage** class, which has a **Storage instance + an index file** that tracks all hints in the form <node : key>.
If we have two hints with the same key but different nodes, a single copy of the object is kept in storage.

Availability criteria

Our system **can always** succeed a client request as long as **all** the following criteria are met.

1. There is at least 1 proxy running;
2. There is at least 1 seed node running;
3. Out of all nodes responsible for the key, at least 1 is available.

(effectively a “minimum viable path” guarantee)

Of course, this is not an ideal situation. There are always downsides to failures. The more nodes and proxies are down, the longer a client request will take to return.

System is scalable as we can add more nodes simply by starting a new instance. Seeds nodes and proxies are harder to scale as they are static.

In CAP terms, our system chooses:

- Availability very aggressively
- Consistency as best-effort
- Partition tolerance is limited

Bottlenecks

- Proxies are a bottleneck, but that's just like any other system employing them.

- **Seed nodes are a bottleneck!** If all seed nodes go down, the system goes down. This is because a proxy only knows about the seed nodes, so it only forwards requests to them.

So technically, seeds are not logically required for coordination, they are just required in practice because proxies only know seed nodes.

How to fix this?

Have proxies learn about non-seeds?
But then will proxy know about the ring??